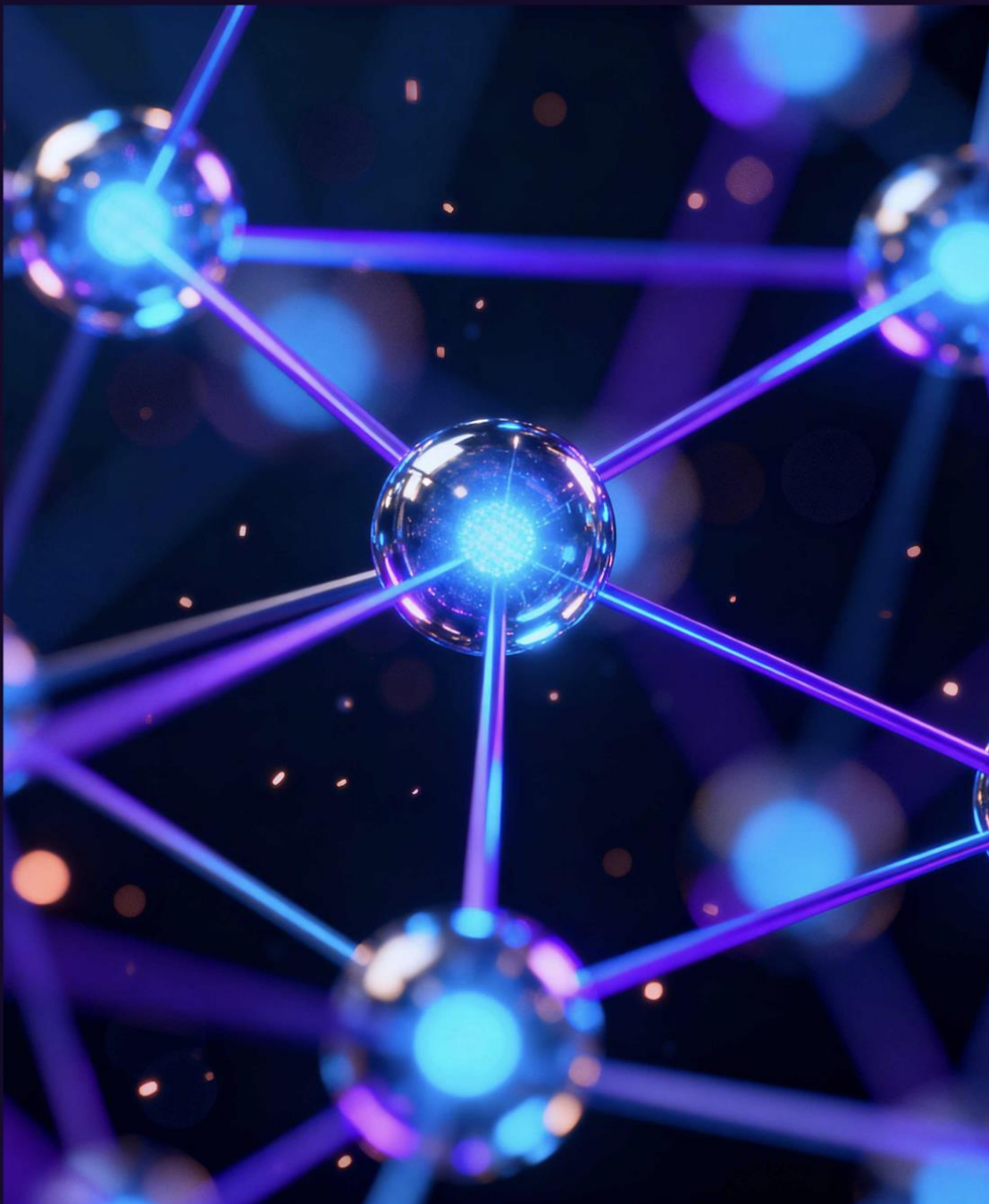




NeuralToolRouter: Scaling Agentic AI Architectures

Optimizing Context Windows and Latency via
Decoupled Tool Retrieval for Production-Ready
Python Implementations

Gurvinder Singh
Senior Solution Architect at IBM

The Challenge: Context Window Bloat

Standard agentic systems send all available tool schemas to the LLM every request, creating bottlenecks that impact costs, latency, and overall model accuracy.

Context Window Bloat

Passing over 100 complex JSON tool schemas per single call consumes massive amounts of input tokens, rapidly filling up the context window limit during execution.

High Latency

Larger context sizes directly increase the Time-to-First-Token (TTFT) and overall inference speed, causing noticeable delays in the agent's real-time responsiveness.

Exorbitant Costs

Input tokens add up extremely quickly for every single request, leading to heavily inflated API bills that drastically reduce the overall profitability of the system.

Degraded Accuracy

LLMs suffer from 'lost-in-the-middle' syndrome; exposing them to irrelevant tools increases hallucination, misrouting, and general errors in complex task reasoning.

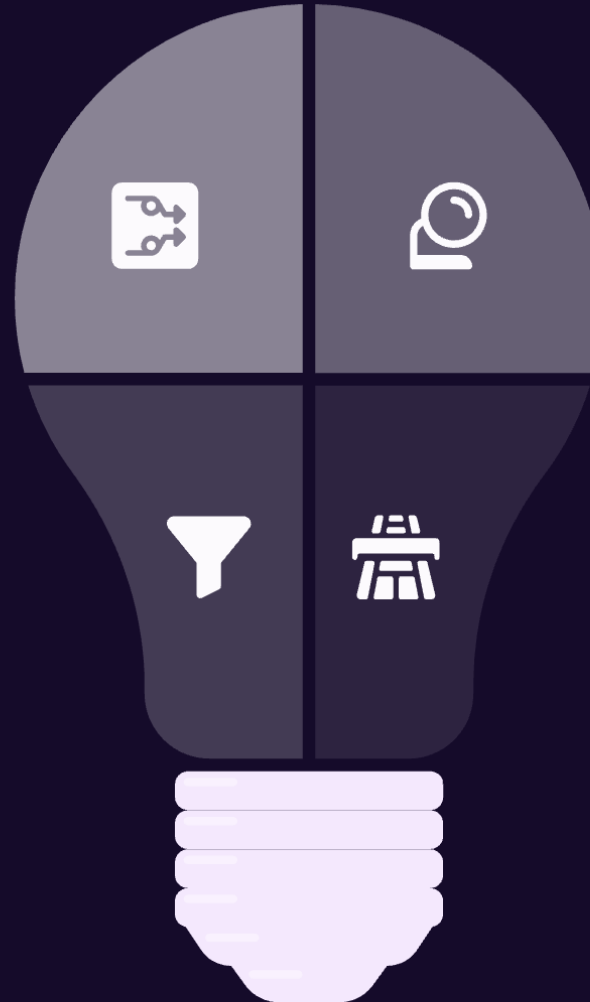
Introducing a "RAG-for-Tools" Architecture

Fast Retrieval

Uses fine-tuned PyTorch embedding models to instantly fetch only the Top-K relevant tools needed for specific user requests, improving processing efficiency.

Reduced Context

Passes only the schemas for the Top-K retrieved tools to the heavy "brain" LLM, ensuring the model remains focused without being overwhelmed by excessive data.



Query Expansion

Employs a lightweight, fast LLM like GPT-4o-mini to expand user intent and refine query clarity before the system initiates the actual tool retrieval process.

Dynamic Fallback

The LLM retains a `search_available_tools` fallback function to self-correct and retrieve additional tools if the initially selected ones do not resolve the request.

Measurable ROI & Scalability: Immediate performance benefits and compounding efficiency gains for complex AI architectures.

90%

Context Reduction

Payload condensed

4.5s

Response Speed

Latency reduction

90%

Cost Savings

Token cost drop

1,000

Tool Scalability

O(1) retrieval

How It Works Under the Hood: Our Architecture Workflow



Query Expander

System breaks down the user's original prompt into specific, actionable logical sub-steps.



Semantic Router

Embeds expanded queries and searches the Vector Index for relevant MCP tool signatures.



Context Assembler

Gathers full JSON schemas for only the Top-K matching tools and the specific fallback tool.



Tool Executor

Executes the chosen tool through the open Model Context Protocol standard for final action.

Built for Production Readiness: Three-Phase Pipeline

01 Synthetic Data Gen

Connects to MCP servers to fetch schemas, leveraging a Teacher LLM to create diverse synthetic user queries specifically tailored

02 Model Fine-Tuning

Trains a lightweight embedding model using contrastive learning on synthetic data, creating a specialized FAISS or ChromaDB

03 Agentic Runtime

Live execution environment utilizing hybrid retrieval, combining Semantic and BM25 methods to route user requests accurately to



Powered by Open Standards: A Future-Proof AI Architecture



Model Context Protocol

Uses MCP by Anthropic to standardize how disparate tools and data sources connect to



FAISS & ChromaDB

Employs high-speed FAISS for CPU/GPU retrieval or ChromaDB to manage scalable vector data.



PyTorch & Transformers

Utilizes sentence-transformers with PyTorch, ensuring CUDA-compatible high-



LiteLLM Integration

Enables seamless switching between OpenAI, Anthropic, Google, and local LLM



Adaptable Architecture for Any Agentic Use Case Requirements

	Priority	Strategy	Components
20 Minimized overhead for speed	Speed	Minimised overhead	paraphrase-MiniLM-L3-v2, Top-K=2, faiss-cpu
40% Deep semantic accuracy	Accuracy	Deep semantic understanding	all-mpnet-base-v2, Top-K=5, Hybrid Retrieval
60% Reduced cloud dependency cost	Cost	Minimal cloud dependency	Open-source expansion LLM, gpt-4o-mini



Drop-in Replacement

- ✗ **Compatible Frameworks**

Integration support is built natively for leading platforms including LangChain, AutoGen, IBM BeeAI, and the CrewAI ecosystem for seamless connectivity.

- ✗ **Ease of Integration**

Streamline your development process by replacing existing ToolNode implementations or standard tool arrays with a simple `SemanticRouter.retrieve_tools()` call.

- ✗ **Implementation**

Pass only the resulting schemas directly into your Agent's prompt context to maintain existing core logic while achieving significantly higher processing efficiency.

Future-Proofing Your Agentic AI: Roadmap and Next Steps



Optimize Tool Retrieval

Shift from passing all tools to dynamically retrieving relevant ones for efficient scaling.



Review Setup Pipeline

Examine the 3-phase pipeline setup provided in the repository to understand the core logic.



Clone neural-tool-router

Start the adoption process by cloning the repository from [sinny777/neural-tool-router](https://github.com/sinny777/neural-tool-router).